



UNIVERSITÀ
di **VERONA**

Machine Learning

LINEAR DISCRIMINANT FUNCTIONS

Cigdem Beyan
A. Y. 2025/2026

LINEAR DISCRIMINANT FUNCTIONS

- The objective of this lecture is to present methods for learning **linear discriminant** functions of the form

$$g(x) = w^T x + w_0 \Leftrightarrow \begin{cases} g(x) > 0 & x \in \omega_1 \\ g(x) < 0 & x \in \omega_2 \end{cases}$$

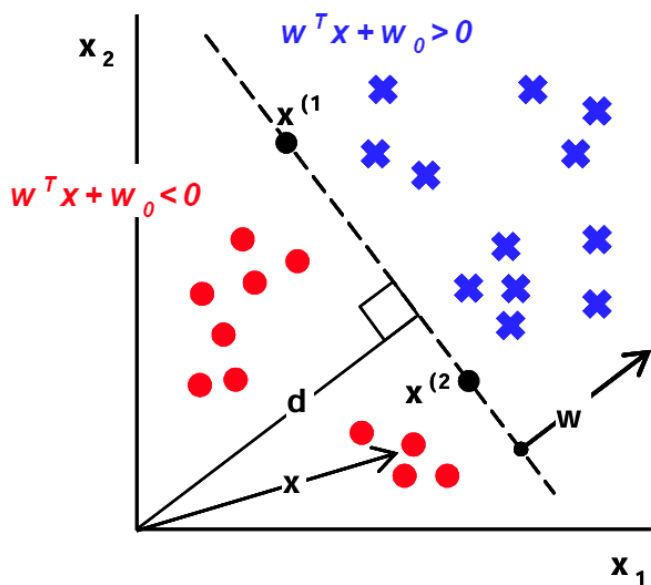
where w is the **weight vector** and w_0 is the **threshold weight** or **bias**.

$$\mathbf{x}=[x_1,\dots,x_n], \mathbf{w}=[w_1,\dots,w_n]$$

- For a problem with 2 classes, the goal is to create **a linear function** $g(x)$ that separates the examples belonging to the two classes.

$$g(x)=0 \rightarrow \text{either class}$$

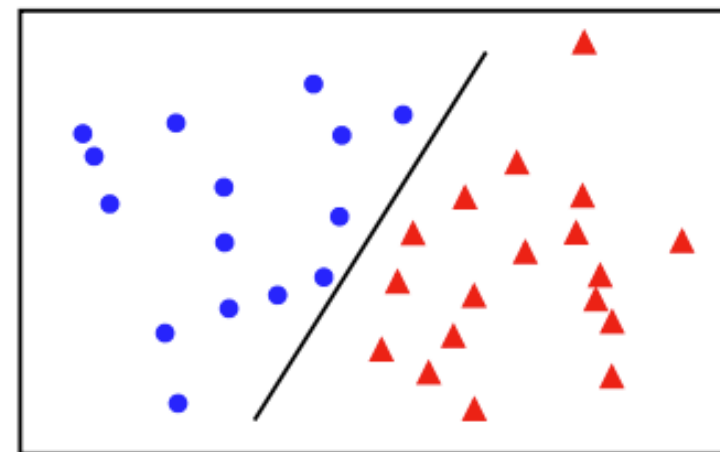
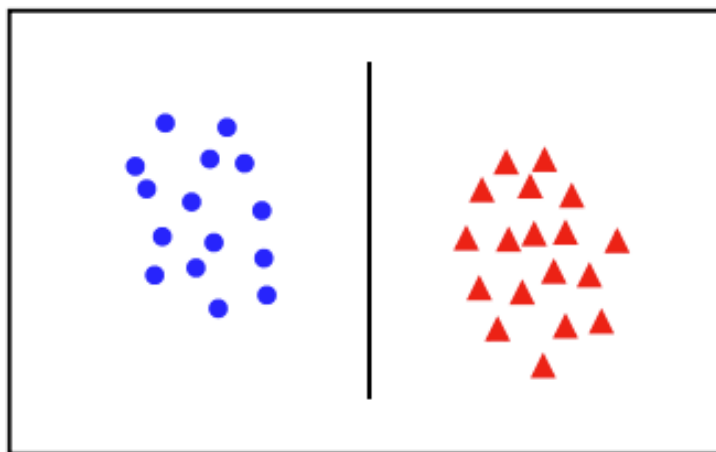
LINEAR DISCRIMINANT FUNCTIONS



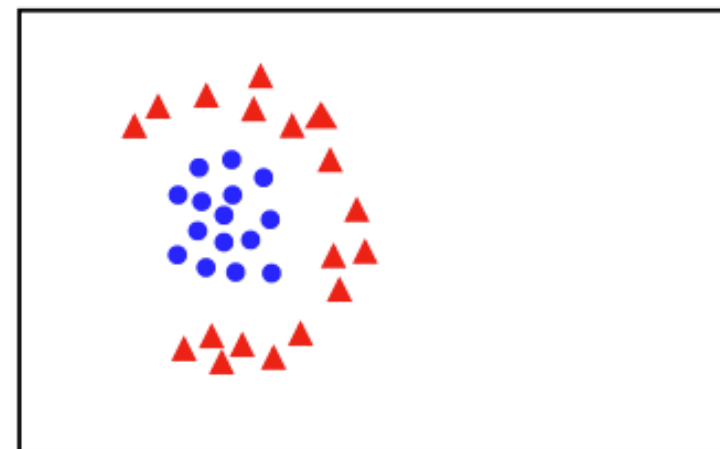
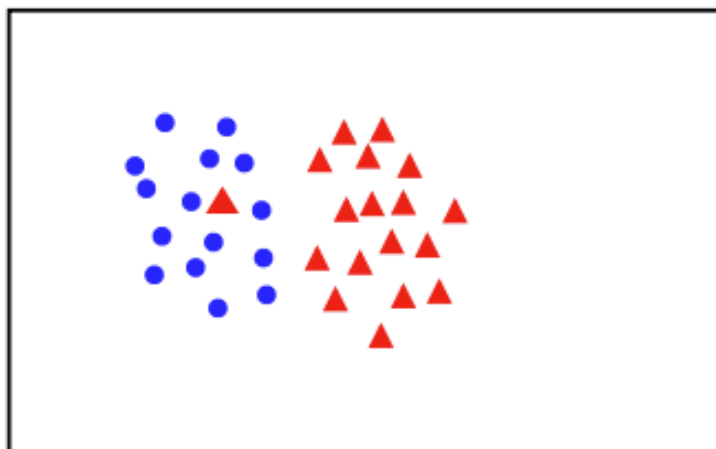
- Suppose we have a set of m samples $x_1, \dots, x_m$, some tagged ω_1 and others tagged ω_2 .
- We want to use these samples to determine the weights $\mathbf{w}$ and w_0 of the linear discriminating function.
- Suppose also that there is a solution for which the probability of error is very low.
- Thus, a reasonable approach is to look for a weight vector that classifies all of the samples correctly or that the probability of making a mistake is zero.
- If such a **weight vector** exists, the samples are said to be *linearly separable*.

LINEAR SEPARABILITY

Linearly
separable

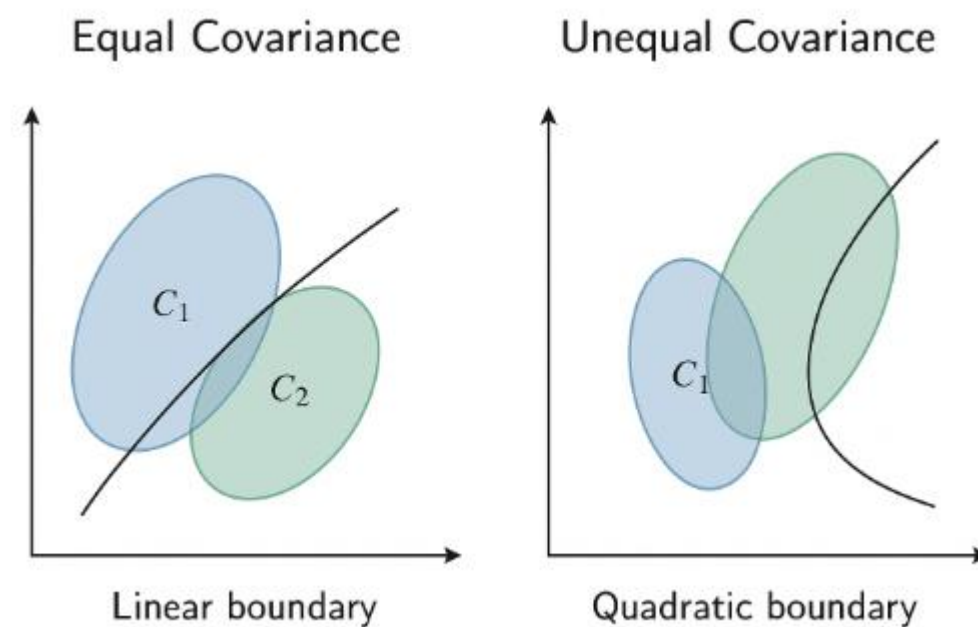


NOT
Linearly
separable



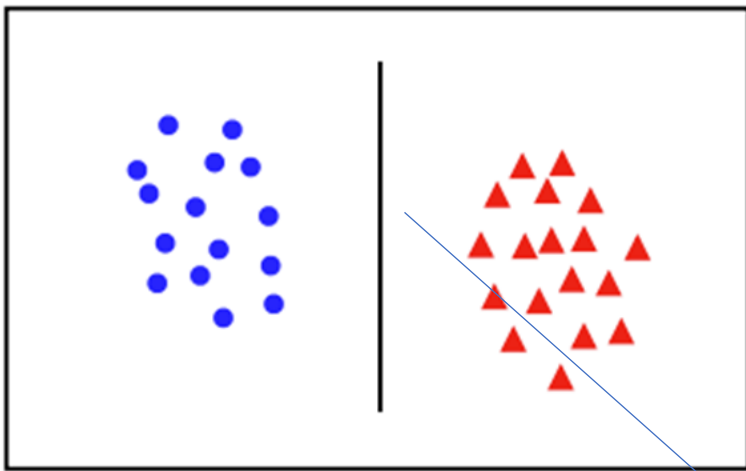
LINEAR DISCRIMINANT FUNCTIONS

- Linear discriminant functions are optimal for Gaussian distributions with equal covariance.
- May not be optimal for other data distributions, but they are very simple to use.
- Knowledge of **class densities** is not required when using linear discriminant functions.
 - Therefore, we can say that this is a **non-parametric approach**



LINEAR CLASSIFIERS

- Every linear discriminant function defines a linear classifier.
- But not every linear classifier is necessarily described in the form of a “discriminant function” (although in practice, the terms are often used interchangeably).

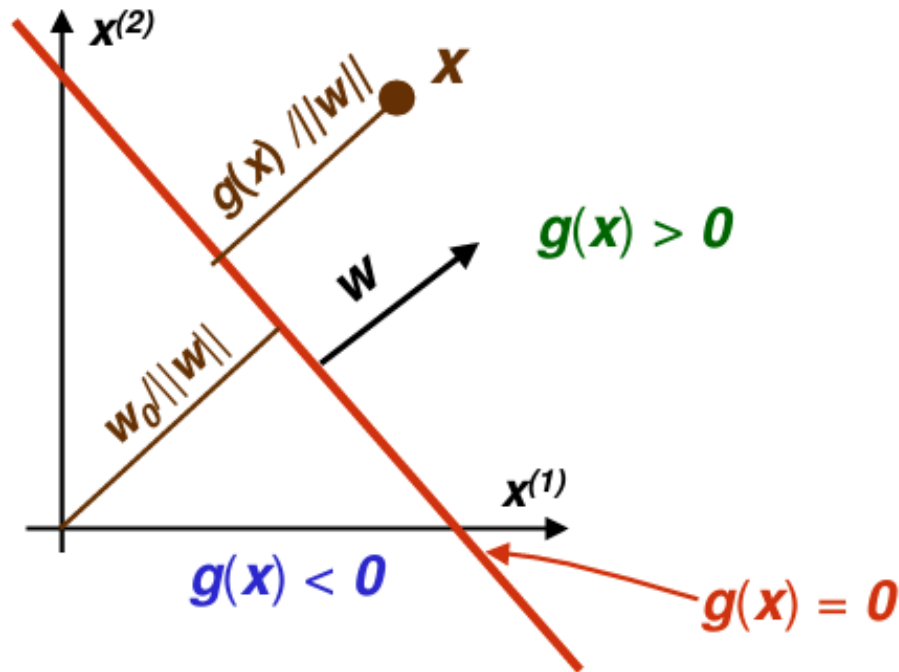


decision
boundary

$$g(x) = w^T x + w_0 \Leftrightarrow \begin{cases} g(x) > 0 & x \in \omega_1 \\ g(x) < 0 & x \in \omega_2 \end{cases}$$

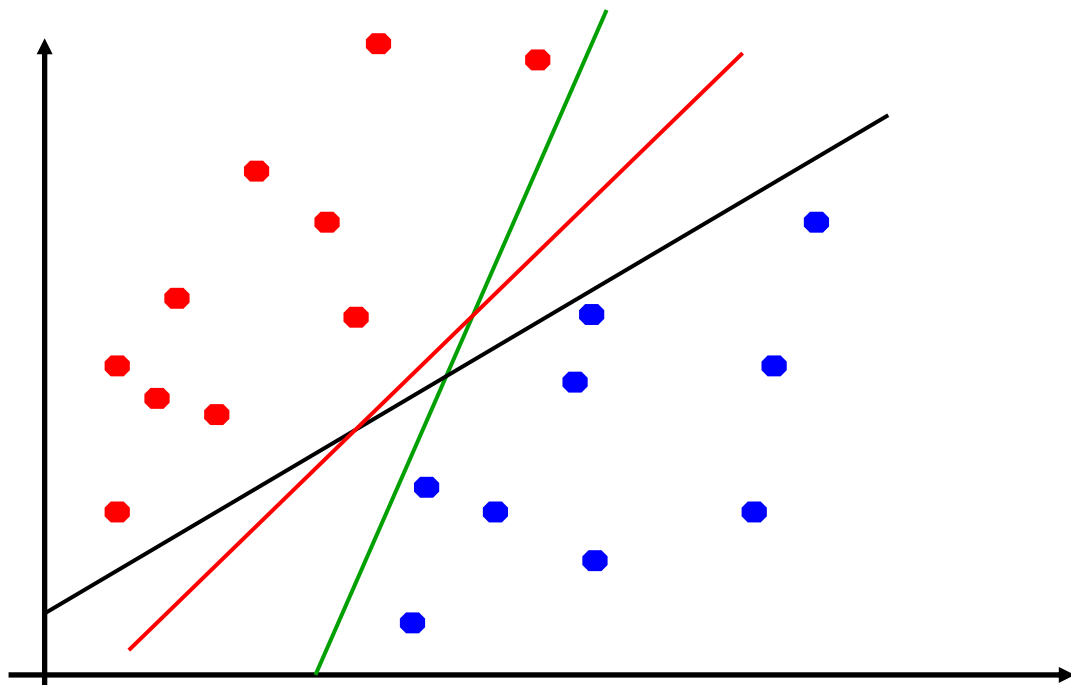
- Decision boundary $g(x) = w^T x + w_0 = 0$
- in 2D, is a line
- in 3D, is a plane
- in nD, is a hyperplane

LINEAR CLASSIFIERS



- The training data is used to learn w .
Only w is needed to classify the new data point.
- The equation $w^t x_i = 0$ defines a hyperplane passing through the origin of the feature space, having w as a normal vector.
- w determines the orientation of the decision hyperplane.
- w_0 determines the location of the decision surface.

WHICH HYPERPLANES?



- If a solution vector exist (if there is a hyperplane) then,
 - Which hyperplane a linear classifier choose when the data is linearly separable?
- The solution vector is not unique!

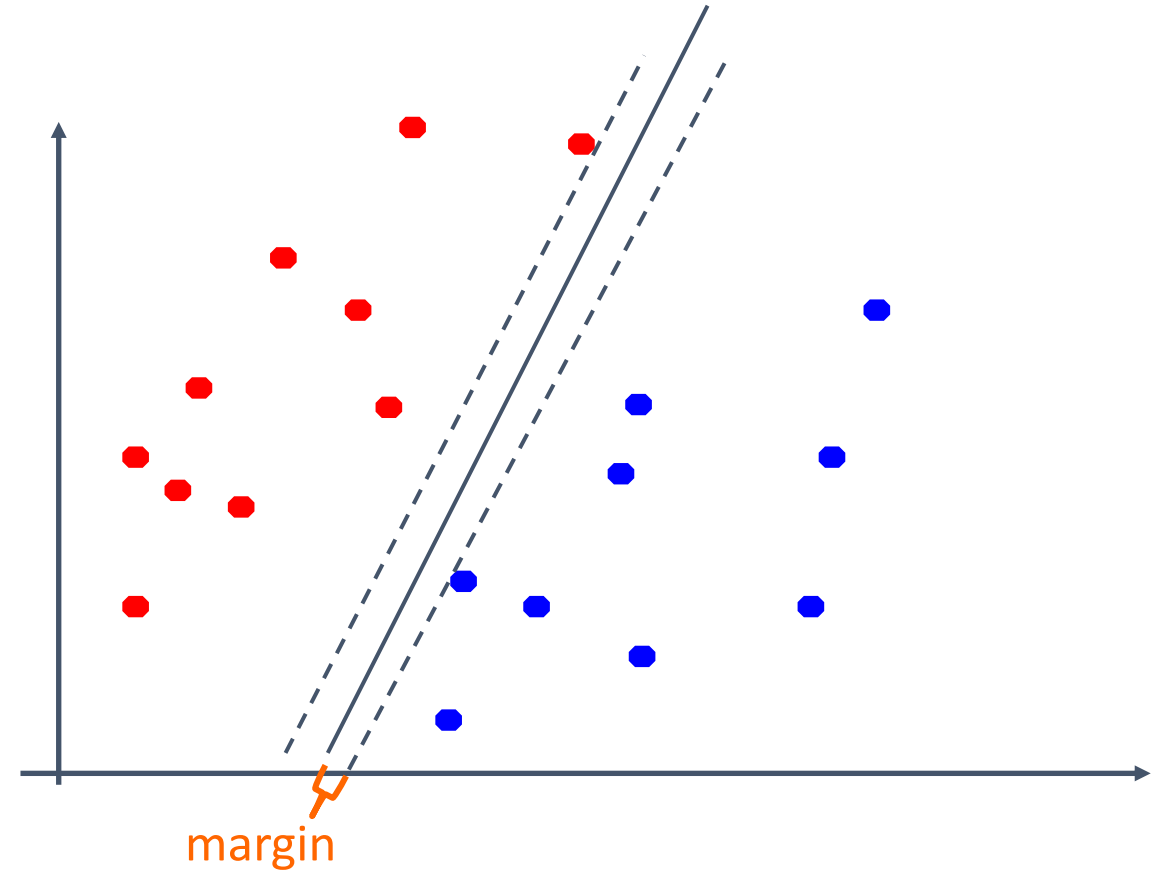
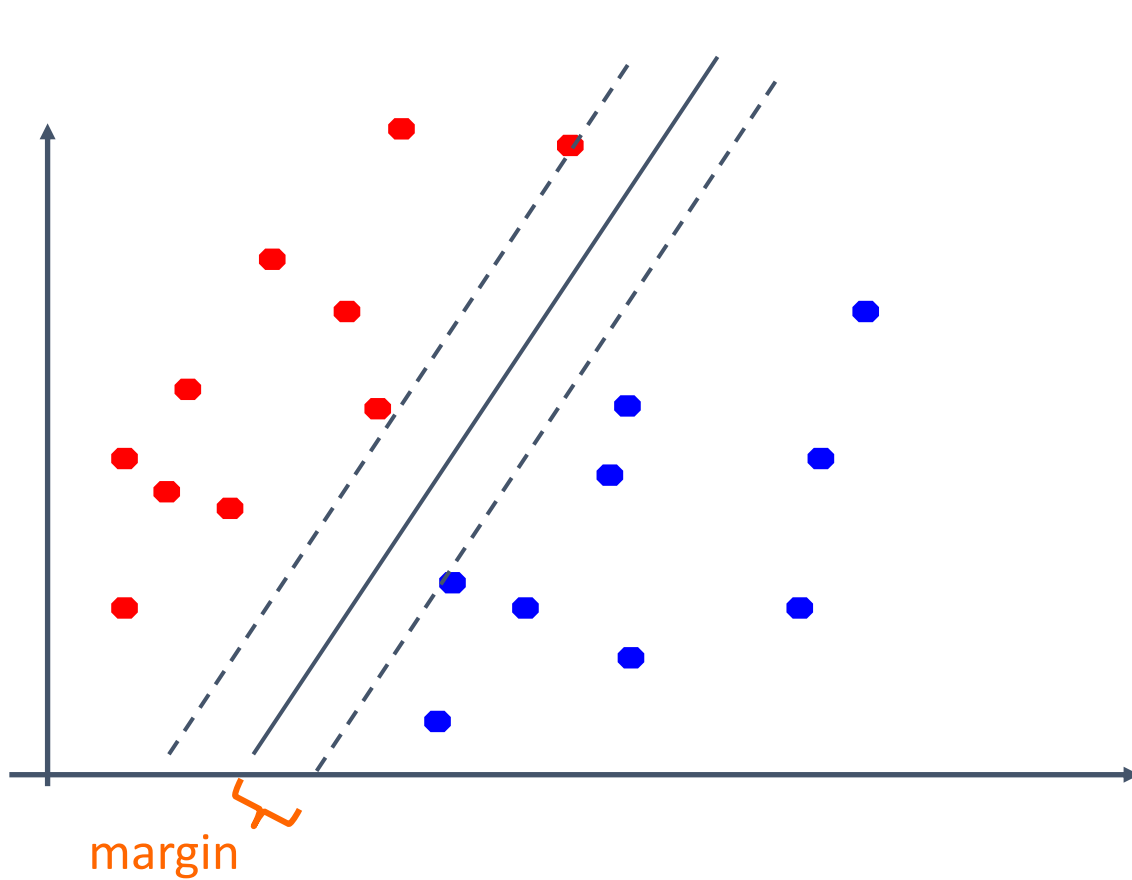
LINEAR DISCRIMINANT FUNCTIONS

- One possibility is to seek a **unit-length weight vector** that **maximizes** the **minimum distance** from **the samples** to the **separating plane**.
- Another possibility is to seek the **minimum-length weight vector** satisfying

$$\mathbf{w}^t \mathbf{x}_i \geq b, \quad \forall i$$

where b is a positive constant called the *margin*.

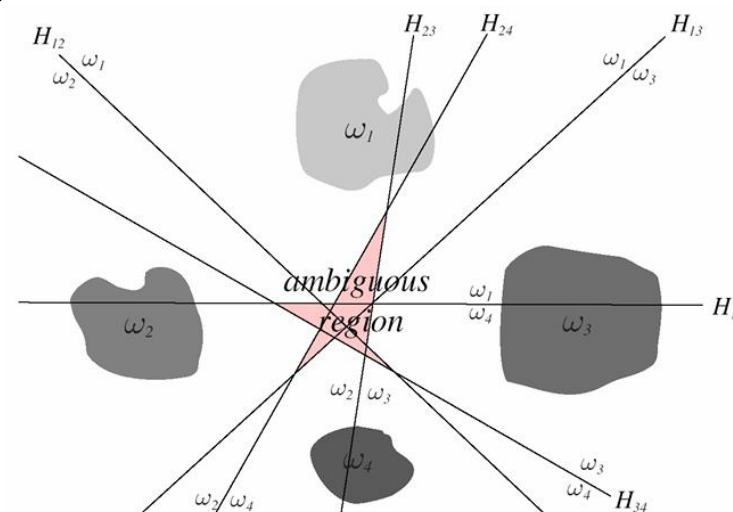
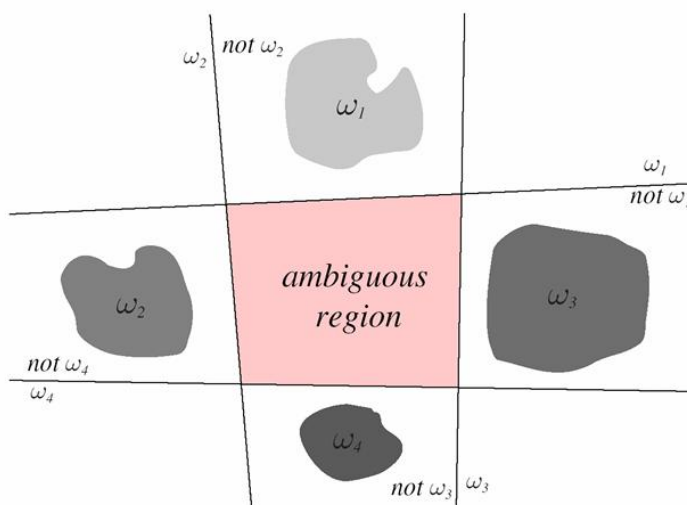
- These techniques attempt to find a solution vector closer to the “**middle**” of the solution region:
 - In this way, the resulting solution is more likely to classify new test samples correctly.



The **margin** of a classifier is the distance to the closest points of either class.

LINEAR DISCRIMINANT FUNCTIONS

- In case of a multi-class (C -class) problem:
 - **One-vs-rest classifiers:** $C-1$ classifiers $g_i(\mathbf{x})$ are constructed, one for each class C_i versus non- C_i .
 - **One-vs-one:** $C(C-1)/2$ binary classifiers (one-vs-one), and then classify a test sample following a **majority voting**.
 - Both lead to areas of **ambiguity**.



LDF: AUGMENTED FEATURE VECTOR

- LDF: $g(x) = w^T x + w_0$
- We can write it: $g(x) = \underbrace{[w_0 \quad w^t]}_{\substack{\text{new weight} \\ \text{vector } a}} \underbrace{\begin{bmatrix} 1 \\ x \end{bmatrix}}_{\substack{\text{new feature} \\ \text{vector } y}} = a^t y = g(y)$

- **y** is called the **augmented feature vector**
- We added a dummy dimension to get a completely equivalent new **homogeneous** problem.

<i>old problem</i>	<i>new problem</i>
$g(x) = w^t x + w_0$	$g(y) = a^t y$
$\begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}$	$\begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \end{bmatrix}$

- Feature augmenting is done only for simpler notation.
- Throughout these lecture notes we replace **a** with **w** for simplicity, since they both refer to *weights*.

LDF: TRAINING ERROR

- For the rest of the lecture, assume we have 2 classes.
- We will use samples (training data) to determine weights $\mathbf{a}$ (or indicated with $\mathbf{w}$) in the discriminant function $\mathbf{g}(\mathbf{y}) = \mathbf{a}^T \mathbf{y}$
- What should be our criterion for determining $\mathbf{a}$?
 - Suppose, for now that we want to minimize the training error (the number of misclassified samples $y_1, \dots, y_n$)
- Recall that $g(y_i) > 0 \rightarrow y_i$ classified as class 1
 $g(y_i) < 0 \rightarrow y_i$ classified as class 2
- Thus, training error is 0 if $\left\{ \begin{array}{l} g(y_i) > 0 \quad \forall y_i \in C_1 \\ g(y_i) < 0 \quad \forall y_i \in C_2 \end{array} \right.$

LDF: TRAINING ERROR

- Thus, training error is **0** if

$$\left\{ \begin{array}{l} g(y_i) > 0 \quad \forall y_i \in C_1 \\ g(y_i) < 0 \quad \forall y_i \in C_2 \end{array} \right.$$

$$\left\{ \begin{array}{l} a^t y_i > 0 \quad \forall y_i \in C_1 \\ a^t y_i < 0 \quad \forall y_i \in C_2 \end{array} \right.$$

$$\left\{ \begin{array}{l} a^t y_i > 0 \quad \forall y_i \in C_1 \\ a^t (-y_i) > 0 \quad \forall y_i \in C_2 \end{array} \right.$$

LDF: “NORMALIZATION”

- This suggests a so-called “**normalization**”
 - Replace all examples from class 2 by their negatives

$$y_i \rightarrow -y_i \quad \forall y_i \in C_2$$

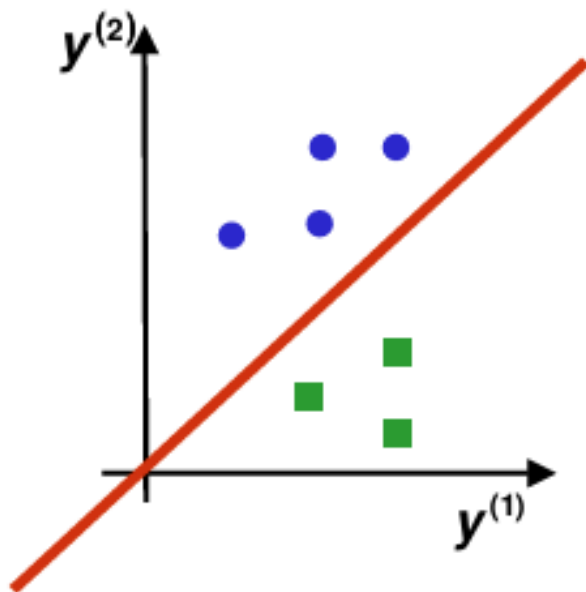
- See weight vector **a** such that

$$a^t y_i > 0 \quad \forall y_i$$

- If such **a** exists, it is called a **separating** or **solution vector**.
 - Original samples $x_1, \dots, x_n$ can indeed be separated by a line in that way.

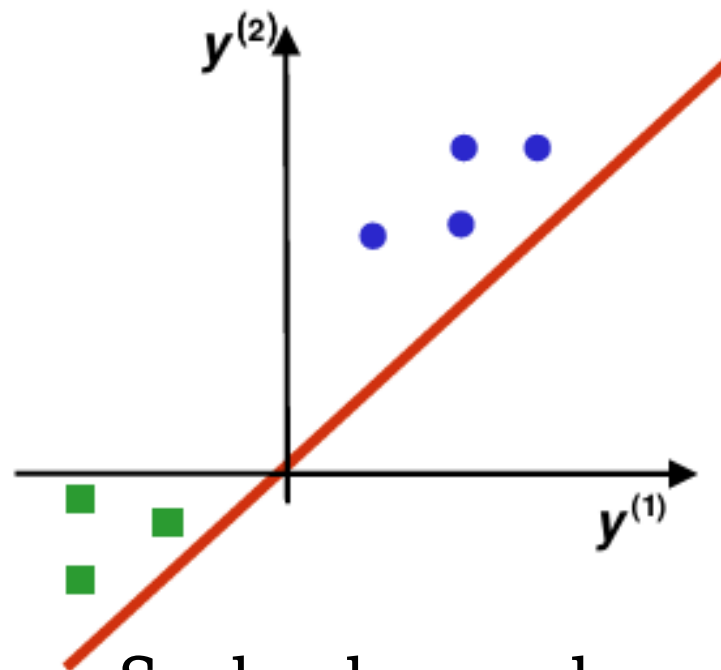
LDF: "NORMALIZATION"

Before normalization



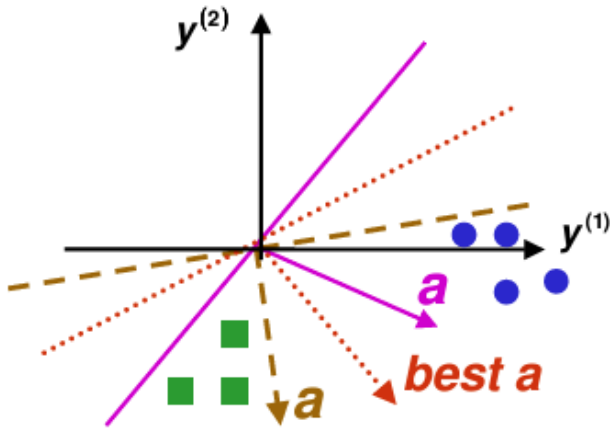
Seek a hyperplane that separates patterns from different categories.

After normalization



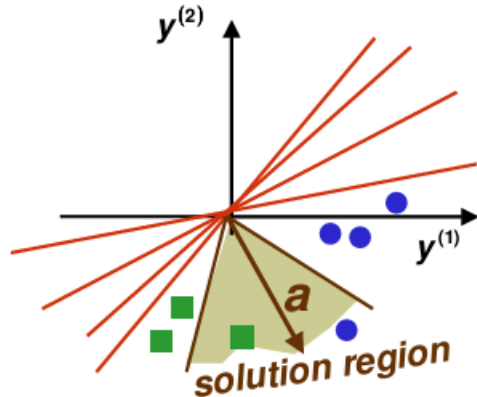
Seek a hyperplane that puts normalized patterns on the same positive side.

LDF: SOLUTION REGION



- Coming back to the “which hyperplane” question....
- We can see that many solutions for the weight vector **a** exist.

$$\mathbf{a}^t \mathbf{y}_i = \sum_{k=0}^d \mathbf{a}_k \mathbf{y}_i^{(k)} > 0$$

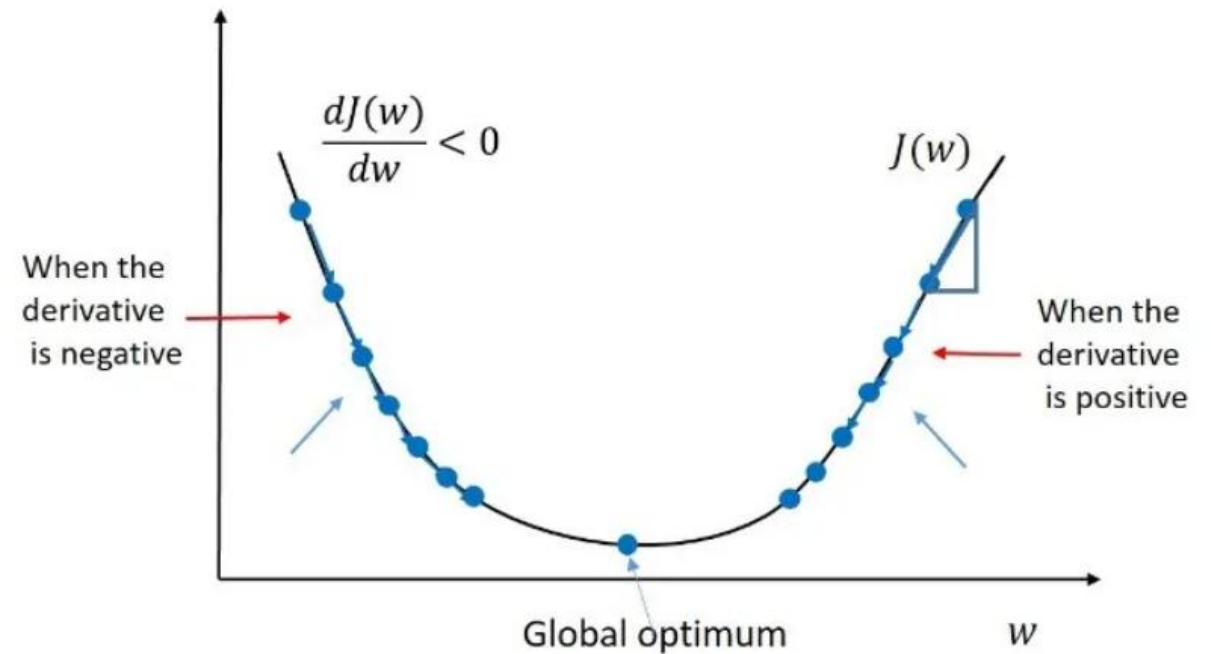


- The **solution region** corresponding to **a** consists of all points classified on one side of the hyperplane, where the hyperplane is **perpendicular** to the **normal vector a**."

GRADIENT DESCENT

- **Gradient Descent** is one of the simplest approaches to calculating a discriminant function.
- It is an iterative method for progressively adjusting the **weights**.

*The gradient vector in space W points to the direction of **maximum deviation of a function** to be maximized/minimized.*



GRADIENT DESCENT

- The procedure consists of updating the value of the weight vector at the **$k+1$ step** with a contribution proportional to the gradient module calculated at the previous step:

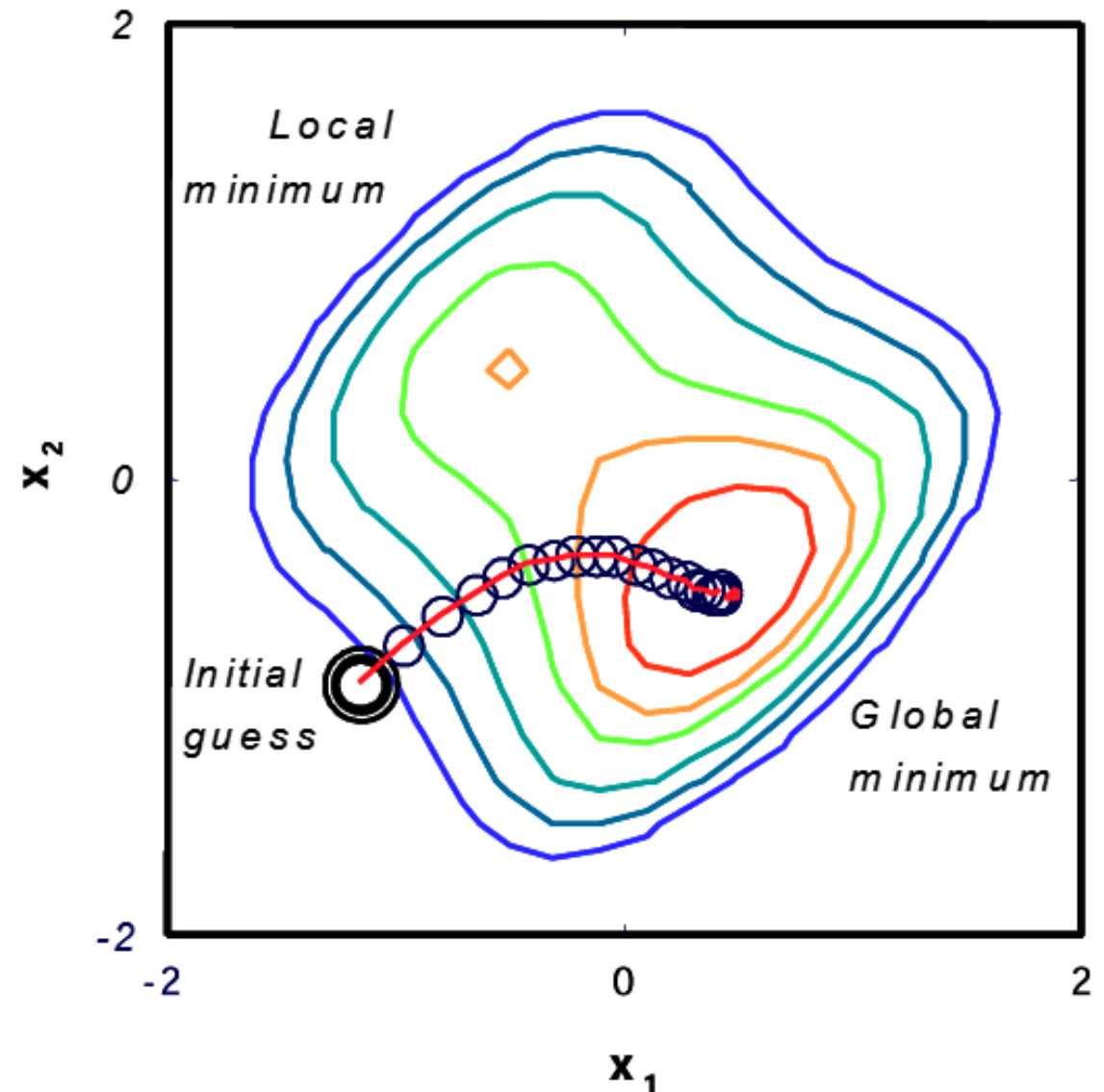
$$\mathbf{w}(k+1) = \mathbf{w}(k) - \rho_k \nabla J(\mathbf{w}) \Big|_{\mathbf{w}=\mathbf{w}(k)}$$

where **$J(\mathbf{w})$** is an **evaluation function** (*loss function*) that must be **minimized**.

- **$J(\mathbf{w})$** is chosen in such a way as to reach **the minimum** when **$\mathbf{w}$** approaches the **optimal solution**;
 - that is, **$J(\mathbf{w})$** should be convex, and **$\mathbf{w}$** is the solution vector.

GRADIENT DESCENT

- The minimum of $J(\mathbf{w})$ is obtained by moving $\mathbf{w}$ in the direction opposite to the gradient (*in the direction of steepest descent*).
- Gradient descent is guaranteed to find only a **local minimum**.
- Very popular, as it is simple and applicable to any function.



GRADIENT DESCENT

- ∇ is the gradient operator symbol, given by:

$$\nabla J = \begin{bmatrix} \frac{\partial J}{\partial w_1} \\ \vdots \\ \frac{\partial J}{\partial w_n} \end{bmatrix}$$

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \rho_k \nabla J(\mathbf{w})|_{\mathbf{w}=\mathbf{w}(k)}$$

- ρ_k (**learning rate**) is an appropriate scalar value that varies with the iteration k , fixing the "magnitude" of the correction.



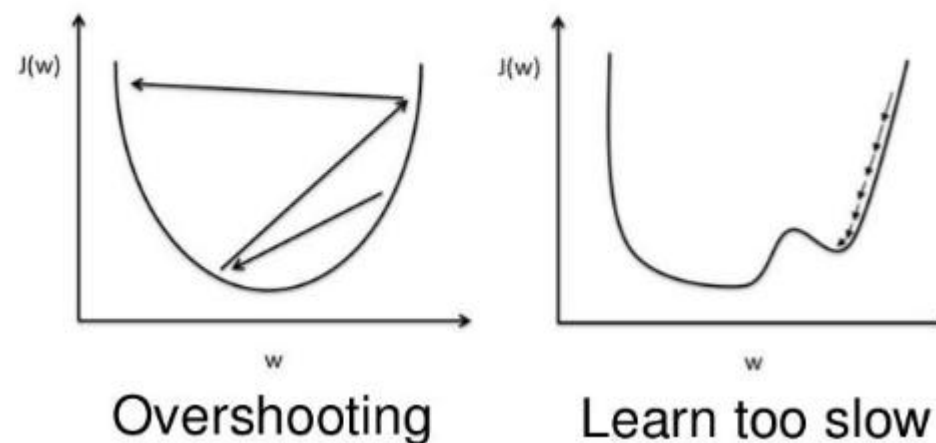
GRADIENT DESCENT

- To summarize:
- At the k -th iteration:
 - Start with a chosen weight vector $\mathbf{w}(1)$ and calculate its gradient $\mathbf{J}(\mathbf{w}(1))$.
 - Then, move from $\mathbf{w}(1)$ in the direction of the steepest descent (opposite to the gradient) by a fixed step size ρ_k to obtain the new vector $\mathbf{w}(2)$.
- **Main issue:** How to determine the optimal choice of ρ_k

GRADIENT DESCENT

- **Main issue:** How to determine the optimal choice of ρ_k
- If ρ_k is too **large**, gradient descent may overshoot the minimum and possibly never find the minimum.
- If ρ_k is too **small**, it needs too many iterations.

Learning rate



PERCEPTRON

- Let's learn a **binary classifier** with linear discriminant function.

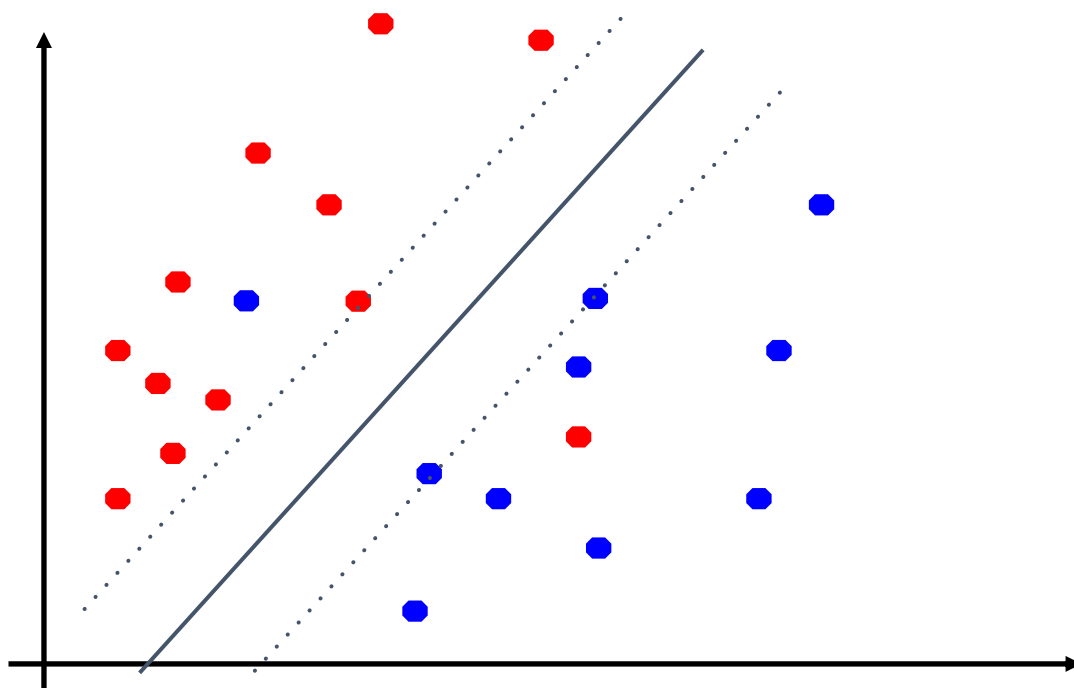
!!!! From this point to the rest $\mathbf{w}$ and $\mathbf{x}$ are used for an augmented feature vector for simplicity, instead of using $\mathbf{a}$ and $\mathbf{y}$.

- To do so, we need to choose a $J(\mathbf{w})$ such that the gradient descent can solve it, in this way we can find the minimum of $J(\mathbf{w})$.
- A good choice is **Perceptron criterion function**: $J(\mathbf{w}) = -\sum_{i \in X} \mathbf{w}^t \mathbf{x}_i$
where X is a set of samples misclassified by $\mathbf{w}$.

$J(\mathbf{w})$ is non-negative since $\mathbf{w}^t \mathbf{x}_i < 0$ for all misclassified samples.

- Do not forget to “normalize” the training set by replacing all examples from class ω_2 with their negatives.

PERCEPTRON



We want to use gradient descent to solve the $J(\mathbf{w})$ so that it can converge; however, for such data (not linearly separable), we cannot converge.

PERCEPTRON

- To find the minimum of $J(\mathbf{w})$, we use gradient descent, which is defined by:
 - i -th component of the gradient of $J(\mathbf{w})$ is equal to $\partial J / \partial w_i$, thus

$$J(\mathbf{w}) = -\sum_{i \in X} \mathbf{w}^t \mathbf{x}_i \quad \Rightarrow \quad \nabla J = -\sum_{i \in X} \mathbf{x}_i$$

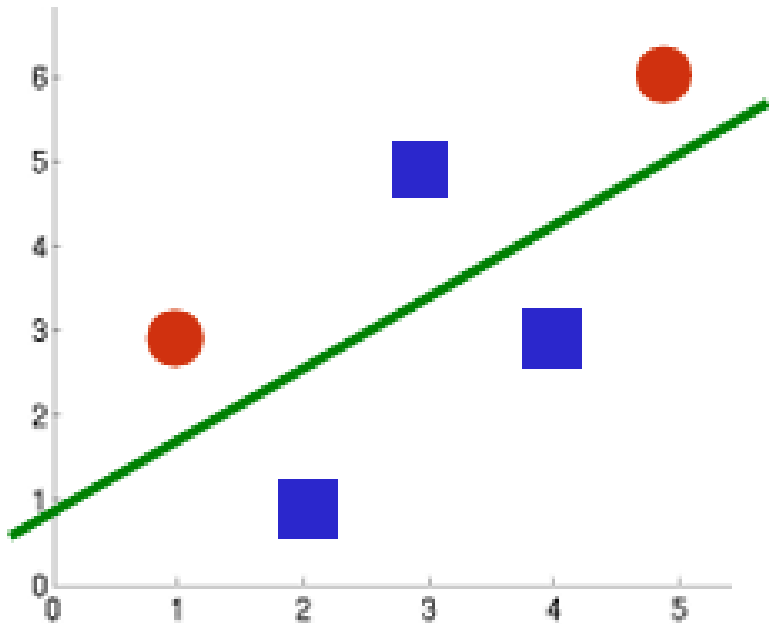
Note that this does not depend on $\mathbf{w}$

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \rho_k \cdot \sum_{i \in X} \mathbf{x}_i, \quad \text{when} \quad \rho_k = \frac{1}{k}$$

Perceptron rule

- Perceptron is the first **Neural Network**, which was then evolved to more complex versions such as **Multilayer Perceptron**.

PERCEPTRON – EXAMPLE (NON-SEPARABLE SCENARIO)



- Suppose we have 2 features and 5 samples for 2 classes.
- Class 1: [2,1], [4,3], [3,5]
- Class 2: [1,3], [5,6]
- These samples are not separable with a line, still we will try to get approximate separation by a line.
- Obtain the augmented features and normalize the data, by doing so we obtain:

$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \\ 3 \end{bmatrix} \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix} : \begin{bmatrix} -1 \\ -1 \\ -3 \end{bmatrix} \begin{bmatrix} -1 \\ -5 \\ -6 \end{bmatrix}$$



PERCEPTRON – EXAMPLE (NON-SEPARABLE SCENARIO)

- Let's apply the perceptron method
 - Initial weights $[1 \ 1 \ 1]$
 - This is a line $x_1 + x_2 + 1 = 0$
- Fixed learning rate $\mathbf{p}(k) = 1$

$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \quad \begin{bmatrix} 1 \\ 4 \\ 3 \end{bmatrix} \quad \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix} : \begin{bmatrix} -1 \\ -1 \\ -3 \end{bmatrix} \quad \begin{bmatrix} -1 \\ -5 \\ -6 \end{bmatrix}$$

$$w_1^t x_1 \rightarrow [1 \ 1 \ 1][1 \ 2 \ 1]^t > 0$$

$$w_1^t x_2 \rightarrow [1 \ 1 \ 1][1 \ 4 \ 3]^t > 0$$

$$w_1^t x_3 \rightarrow [1 \ 1 \ 1][1 \ 3 \ 5]^t > 0$$

$$w_1^t x_4 \rightarrow [1 \ 1 \ 1][-1 \ -1 \ -3]^t = -5 < 0$$

$$\text{Update } w!!! \quad w_2 = [1 \ 1 \ 1] + [-1 \ -1 \ -3] = [0 \ 0 \ -2]$$

$$w_2^t x_5 \rightarrow [0 \ 0 \ -2][-1 \ -5 \ -6]^t > 0$$



PERCEPTRON – EXAMPLE (NONSEPARABLE SCENARIO)

$$w_2^t x_5 \rightarrow [0 \ 0 \ -2] [-1 \ -5 \ -6]^t > 0$$

$$\text{Continue iteration for } w_2^t x_1 \rightarrow [0 \ 0 \ -2] [1 \ 2 \ 1] < 0$$

Update w again!!!

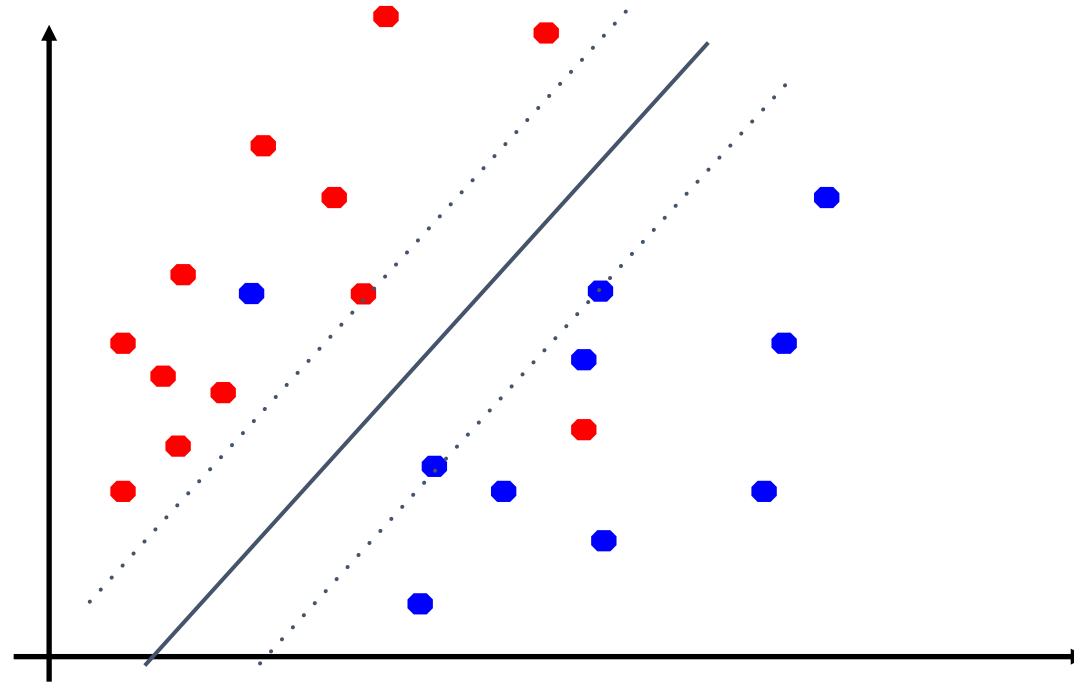
$$w_3 = [0 \ 0 \ -2] + [1 \ 2 \ 1] = [1 \ 2 \ -1]$$

.....

.....

We can continue this forever since there is no solution vector $\mathbf{w}$ satisfying the perceptron rule.

- We want to learn something like this, but our Perceptron constraint does not allow!
- Solution: let's apply some relaxation



PERCEPTRON -- SUMMARY

- If classes are linearly separable, the perceptron rule is guaranteed to converge to a valid solution
 - Some versions of the perceptron rule use a **variable learning rate $p(k)$** , for now, we fixed it as $1/k$
- However, if the two classes are **not linearly separable**, the perceptron rule will **not converge**
 - Since no weight vector $\mathbf{w}$ can correctly classify every sample in a non-separable dataset, the corrections in the perceptron rule will never cease.
 - One ad-hoc solution to this problem is to enforce convergence by using variable learning rates $p(k)$ that approach zero as $k \rightarrow \infty$
 - We will see once we introduce NN architectures.

THE RELAXATION METHOD (1)

- The $J(\mathbf{w})$ takes the form $J_q(\mathbf{w}) = \sum_{i \in X} (\mathbf{w}^t \mathbf{x}_i)^2$
- Also, this function takes into account the **misclassified samples**, and it is minimised when $\mathbf{w}$ is the solution vector.
- Its main difference is that its **gradient is now continuous**, whereas the gradient of the perceptron function J is not.
- So, such a function is suitable for finding smooth surfaces, but this may lead to problems.
 - In fact, in this way, the points very far from the origin $(0,0,\dots,0)$ (*i.e., with large feature magnitudes == outliers*) are weighted a lot.

RELAXATION METHOD (2)

- Another option for $J(\mathbf{w})$ is:
$$J(\mathbf{w}) = \frac{1}{2} \sum_{i \in X} \left\{ \frac{(\mathbf{w}^t \mathbf{x}_i - b)^2}{\|\mathbf{x}_i\|^2} \right\}$$

where $b \geq 0$ margin (e.g., $b = 1$) and $\mathbf{w}^t \mathbf{x}_i \leq b$

- Dividing by $\|\mathbf{x}_i\|^2$ ensures that samples **contribute roughly equally**, regardless of their distance from the origin.

- The gradient descent becomes:
$$\mathbf{w}_{k+1} = \mathbf{w}_k - \rho_k \sum_{i \in X} \left\{ \frac{(\mathbf{w}^t \mathbf{x}_i - b) \mathbf{x}_i}{\|\mathbf{x}_i\|^2} \right\}$$

with $\mathbf{x}_i$ that verify $\mathbf{w}^t \mathbf{x}_i \leq b$ and $\mathbf{w}$ is calculated with the usual rule.

$J(\mathbf{w}) > 0$ always, and $J(\mathbf{w}) = 0$ if $\mathbf{w}^t \mathbf{x}_i \geq b \quad \forall \mathbf{x}_i$



MINIMUM SQUARED ERROR (MSE) SOLUTION

- The classical MSE criterion provides an alternative to the perceptron rule.
 - The perceptron rule only considers **misclassified samples**, since these are the only ones that violate the $J(w)$.
- Instead, the MSE criterion looks for a solution involving **all of the samples** (*not just the misclassified ones*).
 - The goal is to verify $w^t x_i = b_i$, where the b_i are some arbitrarily specified **positive constants**.

MINIMUM SQUARED ERROR (MSE) SOLUTION

- We have replaced the problem of finding the solution to a set of linear inequalities (*perceptron*), now, we have the problem of finding the solution for a set of linear equations (*MSE*).
 - **Perceptron:** $w^t x_i > 0$ for all samples x_i , this solves the system of linear inequalities.
 - **MSE:** $w^t x_i = b_i$ for all samples x_i , this solves the system of linear equations.

Aspect	Perceptron Rule	MSE / Relaxation Method
Loss function	Step / misclassification-based	Continuous quadratic (MSE)
Update	Only for misclassified samples	Gradient descent on all samples
Differentiability	Non-differentiable	Differentiable
Surface	Piecewise constant	Smooth (quadratic)



MINIMUM SQUARED ERROR (MSE) SOLUTION

- MSE procedure
 - Choose positive constants $b_1, b_2, \dots, b_n$
 - Try to find a weight vector $\mathbf{w}$ such that $\mathbf{w}^t \mathbf{x}_i = b_i$ for all samples x_i
 - If we can find a weight vector $\mathbf{w}$ such that $\mathbf{w}^t \mathbf{x}_i = b_i$ for all samples x_i , then w is a solution because b_i 's are all positive.
 - Consider all the samples

MINIMUM SQUARED ERROR (MSE) SOLUTION

- For consistency, we will continue assuming that examples from class ω_2 have been replaced by their **negative vector**, although this is not a requirement for the MSE solution. Thus, we have,

$$x_{i,n+1} = 1 \text{ for } \omega_1 \text{ and } x_{i,n+1} = -1 \text{ for } \omega_2$$

- Suppose we have the following dataset:

$$1^{\text{th}} \text{ sample: } x_{1,1}w_1 + x_{1,2}w_2 + \dots + x_{1,n}w_n + x_{1,n+1}w_{n+1} = b_1, \quad b_1 \geq 0$$

$$2^{\text{th}} \text{ sample: } x_{2,1}w_1 + x_{2,2}w_2 + \dots + x_{2,n}w_n + x_{2,n+1}w_{n+1} = b_2, \quad b_2 \geq 0$$

.....

$$N\text{-th sample: } x_{N,1}w_1 + x_{N,2}w_2 + \dots + x_{N,n}w_n + x_{N,n+1}w_{n+1} = b_N, \quad b_N \geq 0$$

MINIMUM SQUARED ERROR (MSE) SOLUTION

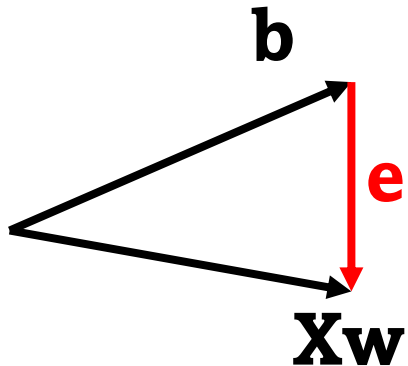
- We then obtain N equations (equal to the training samples, N_1 for ω_1 and N_2 for ω_2), with $N = N_1 + N_2$.
- We need to solve N equations.
- In matrix notation we have: $\mathbf{X} \cdot \mathbf{w} = \mathbf{b}$ when

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,n} & \pm 1 \\ x_{2,1} & \ddots & & \vdots & \vdots \\ \vdots & & \ddots & \vdots & \vdots \\ x_{N,1} & \cdots & \cdots & x_{N,n} & \pm 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_n \\ w_{n+1} \end{bmatrix}$$

- Thus, we need to solve a linear system: $\mathbf{X} \cdot \mathbf{w} = \mathbf{b}$

MINIMUM SQUARED ERROR (MSE) SOLUTION

- Generally, $N \gg n+1$, so there are more equations than unknowns.
 - $\mathbf{X} \cdot \mathbf{w} = \mathbf{b} \rightarrow \mathbf{w} = \mathbf{X}^{-1} \mathbf{b}$
- Let's now inject the **error vector** (needed to obtain the loss function), defined as: $\mathbf{e} = \mathbf{X} \mathbf{w} - \mathbf{b}$ (*imagine this as b is the real class label and Xw is your predictions*)
- MSE finds w , which minimises the length of the error vector $\mathbf{e}$. Thus, minimise the **minimum squared error** criterion function $J(w)$:



$$J_M(\mathbf{w}) = (\mathbf{X} \mathbf{w} - \mathbf{b})^t (\mathbf{X} \mathbf{w} - \mathbf{b}) = \| \mathbf{X} \mathbf{w} - \mathbf{b} \|^2$$



MINIMUM SQUARED ERROR (MSE) SOLUTION

- Unlike the perceptron criterion, the MSE function is quadratic and differentiable.
 - Therefore, we can find its minimum **analytically** by setting the **gradient to zero**.
- The gradient of the objective function $J_M(\mathbf{w})$:
 - Find derivative of $J_M(\mathbf{w})$ wrt. to w .
- Given $J_M(\mathbf{w}) = (\mathbf{X} \mathbf{w} - \mathbf{b})^t (\mathbf{X} \mathbf{w} - \mathbf{b}) = \| \mathbf{X} \mathbf{w} - \mathbf{b} \|^2$ in matrix format, we differentiate the expression $(\mathbf{X} \mathbf{w} - \mathbf{b})^t (\mathbf{X} \mathbf{w} - \mathbf{b})$ with respect to w . Let's denote $\mathbf{Y} = \mathbf{X} \mathbf{w} - \mathbf{b}$, then $J_M(\mathbf{w}) = \mathbf{Y}^t \mathbf{Y}$

MINIMUM SQUARED ERROR (MSE) SOLUTION

Let's denote $Y = \mathbf{X} \mathbf{w} - \mathbf{b}$, then $J_M(\mathbf{w}) = Y^T Y$

Now, differentiating $J_M(\mathbf{w})$ wrt. $\mathbf{w}$, we have:

$$\begin{aligned} \frac{dJ_M(w)}{dw} &= \frac{d}{dw} (Y^T Y) \\ &= 2Y^T \frac{dY}{dw} \quad \rightarrow \quad \frac{dY}{dw} = \frac{d}{dw} (Xw - b) = X \end{aligned}$$

Remember
the chain
rule, if not
clear to you

Therefore, $\frac{dJ_M(w)}{dw} = 2Y^T X = 2(Xw - b)^T X$

MINIMUM SQUARED ERROR (MSE) SOLUTION

$$\frac{dJ_M(w)}{dw} = 2Y^T X = 2(Xw - b)^T X$$

- By setting the **gradient to zero**, this becomes

$$2(\mathbf{X}\mathbf{w} - \mathbf{b})^t \mathbf{X} = 0 \Rightarrow \mathbf{X}^t \mathbf{X} \mathbf{w} = \mathbf{X}^t \mathbf{b} // \text{the proof can be found in the next slide}$$

- If we multiply by $[\mathbf{X}^t \mathbf{X}]^{-1}$, we get $\mathbf{w} = [\mathbf{X}^t \mathbf{X}]^{-1} \mathbf{X}^t \mathbf{b} = \mathbf{X}^\# \mathbf{b}$
- where $\mathbf{X}^\# = [\mathbf{X}^t \mathbf{X}]^{-1} \mathbf{X}^t$ is the *pseudoinverse* of $\mathbf{X}$. **Pseudo-inverse solution**
- Consequently, $\mathbf{w} = \mathbf{X}^\# \mathbf{b}$ is the optimal solution according to the MSE method.

MINIMUM SQUARED ERROR (MSE) SOLUTION

1 $2(Xw - b)^T X = 0$

2 Since we have a factor of 2 on the left side, divide both sides by 2:

$$(Xw - b)^T X = 0$$

3 The expression $(Xw - b)^T X = 0$ can be rewritten by expanding the transpose:

$$(Xw)^T X - b^T X = 0$$

4 We know that $(Xw)^T X = w^T X^T X$ (since $(AB)^T = B^T A^T$):

$$w^T X^T X - b^T X = 0$$

This equality holds if:

5

$$X^T X w = X^T b$$

This is the desired result, so we have shown that:

$$2(Xw - b)^T X = 0 \Rightarrow X^T X w = X^T b.$$

MSE -- PROPERTIES

- $\mathbf{w} = \mathbf{X}^\# \mathbf{b}$ shows that the MSE solution depends on the margin vector $\mathbf{b}$.
 - Different choices of $\mathbf{b}$ lead to different properties of the solution.
- If $\mathbf{b}$ is arbitrarily fixed, there is no evidence that the MSE solution gives a separator vector in the case of linear separability.
- So, how to choose $\mathbf{b}$ in the MSE procedure?
 - Good choice is $b_1 = b_2 = \dots = b_n = 1$. In this case, the MSE solution is identical to Fischer's linear discriminant solution (*next lesson*).



MSE -- PROPERTIES

- MSE pays too much attention to **outlier examples**!
 - Because MSE squares the error, **large errors (outliers)** have a disproportionately large influence on the solution.
 - For classification, where we only care about *which side* of the boundary a sample is on, this sensitivity is undesirable.

MSE -- PROPERTIES

- $\mathbf{X}^\#$ is difficult to obtain
 - as this requires inverting a large matrix, which is:
 - **expensive** for high-dimensional data
 - **impossible** if $\mathbf{X}^T \mathbf{X}$ is **singular** (non-invertible).
 - The analytical solution is *computationally heavy* and sometimes *ill-defined*.



MSE -- PROPERTIES

- It is not an iterative process, indeed, $\mathbf{w}$ is obtained directly through mathematical computations in **one step**.
 - Even if this sounds good,
 - it means we **lose flexibility**
 - E.g., no gradual correction,
 - online learning

EXAMPLE

- Compute the **perceptron** and **MSE** solution for the dataset

$$X1=[(1,6), (7,2), (8,9), (9,9)]$$

$$X2= [(2,1),(2,2),(2,4),(7,1)]$$

Perceptron learning

- Assume $\rho_k = 0.1$ and an online update rule
 - Assume $w_1 = [0.1, 0.1, 0.1]$
- 1) Normalize the dataset

$$\begin{bmatrix} 1 & 1 & 6 \\ 1 & 7 & 2 \\ 1 & 8 & 9 \\ 1 & 9 & 9 \\ -1 & -2 & -1 \\ -1 & -2 & -2 \\ -1 & -2 & -4 \\ -1 & -7 & -1 \end{bmatrix}$$

EXAMPLE

2) Iterate through all the examples ($\mathbf{w}^t \mathbf{x}_i$) and update $w(k)$

Data point 1: $[0.1 \ 0.1 \ 0.1]^t [1 \ 1 \ 6]$ this is >0 so correctly classified
therefore there is no update

Data point 2: $[0.1 \ 0.1 \ 0.1]^t [1 \ 7 \ 2]$ this is >0 so correctly classified
therefore there is no update

.....

Data point 5: $[0.1 \ 0.1 \ 0.1]^t [-1 \ -2 \ -1]$ this is <0 so we have to apply an
update $w(2)=w(1)+ \rho_1 [-1 \ -2 \ -1] = [0 \ -0.1 \ 0]$

.....



EXAMPLE

Data point 1: $[0 \ -0.1 \ 0]^t [1 \ 1 \ 6]$ this is < 0 so update

$$w(3) = w(2) + \rho_1 [1 \ 1 \ 6] = [0 \ -0.1 \ 0] + 0.1[1 \ 1 \ 6] = [0.1 \ 0 \ 0.6]$$

Data point 2: $[0.1 \ 0 \ 0.6]^t [1 \ 7 \ 2] > 0 \Rightarrow$ no update

The perceptron with this data and setup converges after 175 iterations,
when $w = [-3.5 \ 0.3 \ 0.7]$

EXAMPLE

MSE: This solution will be found in one shot $\mathbf{w} = [\mathbf{X}^t \mathbf{X}]^{-1} \mathbf{X}^t \mathbf{b}$

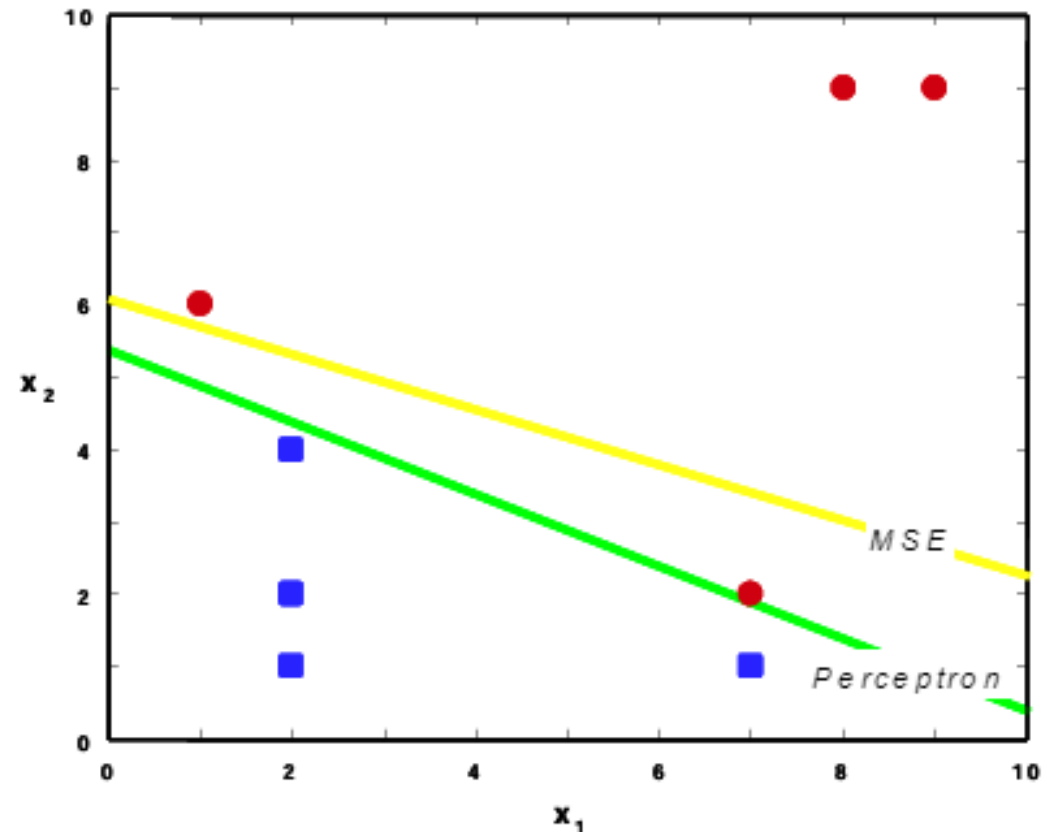
For the choice of $\mathbf{b}=[1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$, the solution is

$$\mathbf{w}=[-1.1870 \ 0.0746 \ 0.1959]$$

$$\mathbf{X}_1=[(1,6), (7,2), (8,9), (9,9)]$$

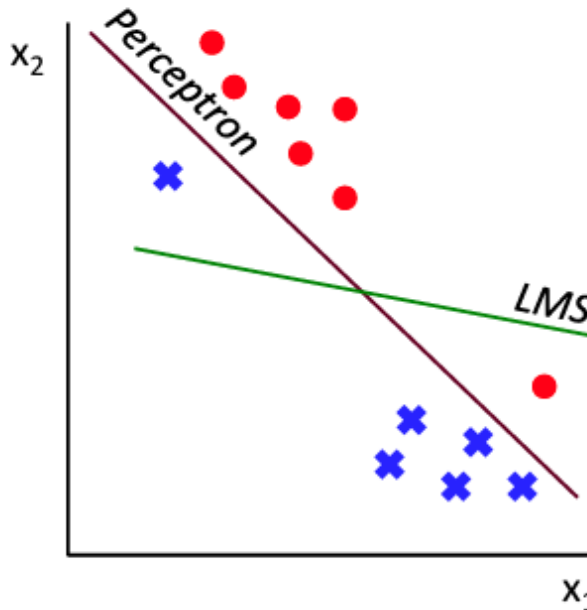
$$\mathbf{X}_2=[(2,1),(2,2),(2,4),(7,1)]$$

- As seen in Fig. The MSE misclassifies one of the samples



PERCEPTRON VS. MSE (SUMMARY)

- **Perceptron rule** –The perceptron rule always finds a solution if the classes are linearly separable, but does not converge if the classes are non-separable
- **MSE criterion:** The MSE solution has guaranteed convergence, but it may not find a separating hyperplane if classes are linearly separable.
 - Notice that MSE tries to minimize the sum of the squares of the distances of the training data to the separating hyperplane, as opposed to finding this hyperplane.



- The objective function $J_M(\mathbf{w})$ can also be minimized using a gradient descent procedure. In this way, we avoid
 - the problems that arise when $\mathbf{X}^t \mathbf{X}$ is singular (*happens if samples are correlated*),
 - the need for working with large matrices.
- Since $\nabla J_M(\mathbf{w}) = 2 \mathbf{X}^t (\mathbf{X}\mathbf{w} - \mathbf{b})$, the gradient descent algorithm is
$$\mathbf{w}_{k+1} = \mathbf{w}_k - \rho_k \mathbf{X}^t (\mathbf{X}\mathbf{w}_k - \mathbf{b})$$
 with $\mathbf{w}_1$ arbitrary,
[X and b are in matrix and vector form, written for all data]
- It can be shown that if $\rho_k = \rho_1/k$ where ρ_1 is any positive constant, this rule generates a sequence of weight vectors that converge to a solution to
$$\mathbf{X}^t (\mathbf{X} \mathbf{w} - \mathbf{b}) = 0$$
 - The gradient descent algorithm leads to the solution regardless of whether $\mathbf{X}^t \mathbf{X}$ is singular or not.



THE LEAST MSE (LMSE = WIDROW-HOFF METHOD)

- Considering **one sample at a time**, we get

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \rho_k (b_k - \mathbf{w}_k^t \mathbf{x}_k) \mathbf{x}_k \quad \text{LMS rule}$$

such that $\mathbf{b}_k$ is the value for the k -th sample of the margin $\mathbf{b}$ ($b_k \geq 0$), i.e., it depends on the sample $\mathbf{x}_k$.

- This method is called Widrow-Hoff, least-mean-squares (LMS) or delta rule
 - reduces **storage requirements** by considering single samples **sequentially**.



THE LEAST MSE (LMSE = WIDROW-HOFF METHOD)

- Widrow-Hoff rule "corrects" $\mathbf{w}_k$ whenever $\mathbf{w}_k^T \mathbf{x}_k \neq b_k$.
- Why?
- The weight change is: $\Delta w_k = w_{k+1} - w_k = \rho_k (b_k - w_k^T x_k) x_k$

Case 1: Prediction is correct

$$w_k^T x_k = b_k$$

then

$$\Delta w_k = \rho_k (b_k - w_k^T x_k) x_k = \rho_k (0) x_k = 0$$

SO $w_{k+1} = w_k \rightarrow$ **no correction happens.**



THE LEAST MSE (LMSE = WIDROW-HOFF METHOD)

Case 2: Prediction is wrong

$$w_k^T x_k \neq b_k$$

then

$$\Delta w_k = \rho_k (b_k - w_k^T x_k) x_k \neq 0$$

SO $w_{k+1} \neq w_k \rightarrow$ **the weight vector is corrected.**

- The advantage of “correcting only when $w_k^T x_k \neq b_k$ ” is that learning is **efficient, stable, proportional, and error-driven**
 - We adjust weights **exactly when and as much as needed.**

WIDROW-HOFF METHOD: SUMMARY

- Iterative, gradient-descent-based update:

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \rho_k (b_k - \mathbf{w}_k^t \mathbf{x}_k) \mathbf{x}_k$$

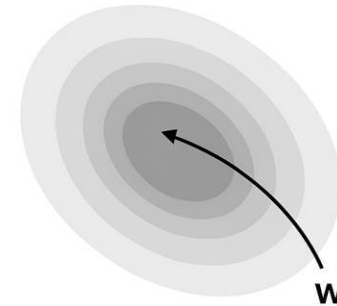
- Can update **sample-by-sample**
 - Low computational cost per iteration, scalable to large datasets
- Learning rate controls convergence speed
- Adapts to new data (suitable for online learning)

WIDROW-HOFF METHOD: DISADVANTAGES

- Can converge slowly
 - Sensitive to correlated features and outliers
- Needs careful learning rate
- **Requires multiple iterations; not one-step like analytical MSE**
- May struggle with non-linear or non-separable data

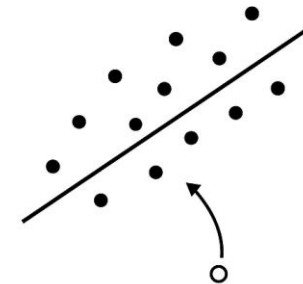
Disadvantages of Widrow-Hoff

Slow convergence



Slow convergence
if features are correlated
or ill-scaled

Sensitivity to outliers



Squared error $\rightarrow$
large influence of
outliers



THE HO-KASHYAP METHOD

- The main limitation of the MSE criterion is the lack of guarantees that a separating hyperplane will be found in the linearly separable case.
- All we can say about the MSE rule is that it minimises $\|\mathbf{X} \mathbf{w} - \mathbf{b}\|^2$ when $\mathbf{b}$ is chosen arbitrarily.

This means that whether MSE finds a separating hyperplane or not depends on how properly the margin $\mathbf{b}$ is selected (*notice that in some sources b is referred to as target outputs*).

THE HO-KASHYAP METHOD

- If the training samples are linearly separable, then there must be vectors $\mathbf{w}^*$ and $\mathbf{b}^*$ such that $\mathbf{X}\mathbf{w}^* = \mathbf{b}^* > 0$ where $\mathbf{b}^* > 0$, meaning that every component of $\mathbf{b}^*$ is **positive**.
- If $\mathbf{b}^*$ were **known**, one could simply use MSE solution to compute separating hyperplane
- Idea: find both $\mathbf{w}^*$ and $\mathbf{b}^*$
- Minimize the following criterion function, restricting to positive **b**

$$J_{HK}(\mathbf{w}, \mathbf{b}) = \| \mathbf{X} \mathbf{w} - \mathbf{b} \|^2$$

- $J_{HK}(\mathbf{w}^*, \mathbf{b}^*) = 0$

THE HO-KASHYAP METHOD

$$J_{HK}(\mathbf{w}, \mathbf{b}) = \| \mathbf{X} \mathbf{w} - \mathbf{b} \|^2$$

- As usual, take partial derivatives with respect to $\mathbf{w}$ and $\mathbf{b}$

$$\nabla_{\mathbf{w}} J_{HK} = -2 \mathbf{W}^T (\mathbf{X} \mathbf{w} - \mathbf{b}) = 0$$

$$\nabla_{\mathbf{b}} J_{HK} = -2 (\mathbf{X} \mathbf{w} - \mathbf{b}) = 0$$

- Use gradient descent to find a minimum $J_{HK}(\mathbf{w}, \mathbf{b})$
- Alternate the two steps below until convergence
 - Fix $\mathbf{b}$ and minimize $J_{HK}(\mathbf{w}, \mathbf{b})$ with respect to $\mathbf{w}$
 - Fix $\mathbf{w}$ and minimize $J_{HK}(\mathbf{w}, \mathbf{b})$ with respect to $\mathbf{b}$

THE HO-KASHYAP METHOD

- The resulting iterative **Ho-Kashyap rule** is:

$$\mathbf{b}_{k+1} = \mathbf{b}_k - \frac{1}{2} \rho_k [\nabla_{\mathbf{b}} J_{HK}(\mathbf{w}_k, \mathbf{b}_k) - |\nabla_{\mathbf{b}} J_{HK}(\mathbf{w}_k, \mathbf{b}_k)|]$$
$$\nabla_{\mathbf{b}} J_{HK} = -2 (\mathbf{X}\mathbf{w} - \mathbf{b}) = 0$$

$$\text{Let } e_k = \mathbf{X}\mathbf{w}_k - \mathbf{b}_k = -\frac{1}{2} [\nabla_{\mathbf{b}} J_{HK}(\mathbf{w}_k, \mathbf{b}_k)]$$

$$\begin{aligned} \text{Then } \mathbf{b}_{k+1} &= \mathbf{b}_k - \frac{1}{2} \rho_k [-2e_k - |2e_k|] \\ &= \mathbf{b}_k + \rho_k [e_k + |e_k|] \end{aligned}$$

THE HO-KASHYAP METHOD

(0) Start with arbitrary $\mathbf{b}_1 > 0$ and $\mathbf{w}_1$, let $k=1$

Repeat steps (1) to (4)

(1) $\mathbf{e}_k = \mathbf{X}\mathbf{w}_k - \mathbf{b}_k$

(2) Solve for $\mathbf{b}_{k+1}$ using $\mathbf{w}_k$ and $\mathbf{b}_k$

$$\mathbf{b}_{k+1} = \mathbf{b}_k + [e_k + |e_k|]$$

(3) Solve for $\mathbf{w}_{k+1}$ using $\mathbf{b}_{k+1}$

$$\mathbf{w}_{k+1} = (\mathbf{w}^T \mathbf{w})^{-1} \mathbf{Y}^T \mathbf{b}_{k+1}$$

(4) $k=k+1$

until $\mathbf{e}_k = 0$ or $k > k_{max}$ or $\mathbf{b}_{k+1} = \mathbf{b}_k$ (for convergence fix learning rate as $0 < \rho_k < 1$)

THE HO-KASHYAP METHOD

- In the linearly separable case,
 - $e_k = 0$, solution is found, STOP!
 - One of the components of e_k is positive, algorithm continues
- In non separable case,
 - e_k will have only negative components eventually, thus found proof of non-seperability.
 - No bound on how mainy iterations is needed for the proof of non-seperability.



SUMMARY

- **Perceptron** Procedure
 - Find a separating hyperplane in the linearly separable case,
 - Do not converge in the non-separable case
 - Can force convergence by using a decreasing learning rate, but there is no guarantee a reasonable stopping point
- **MSE** Procedure
 - Converge in separable and non-separable cases
 - May not find a separating hyperplane if classes are linearly separable
 - Use pseudoinverse if X^tX is not singular and not too large
 - Use gradient descent (Widrow-Hoff procedure) otherwise
- **Ho-Kashyap** Procedure
 - Always converge
 - Find a separating hyperplane in the linearly separable case
 - More costly



That's all

Cigdem Beyan
cigdem.beyan@univr.it
<https://cbeyan.github.io/>